

C++ Standard Library Issues to be moved in St. Louis

Doc. no. P3341R0

Date: 2024-06-24

Audience: WG21

Reply to: Jonathan Wakely <lwgcchair@gmail.com>

- [Ready Issues](#)
- [Tentatively Ready Issues](#)

Ready Issues

[3944](#). Formatters converting sequences of char to sequences of wchar_t

Section: 22.14.6.4 [\[format.formatter.spec\]](#) Status: [Ready](#) Submitter: Mark de Wever Opened: 2023-06-01 Last modified: 2024-03-18

Priority: 3

View other [active issues](#) in [\[format.formatter.spec\]](#).

View all other [issues](#) in [\[format.formatter.spec\]](#).

Discussion:

I noticed some interesting features introduced by the range based formatters in C++23

```
// Ill-formed in C++20 and C++23
const char* cstr = "hello";
char* str = const_cast<char*>(cstr);
std::format(L"{}", str);
std::format(L"{}", cstr);

// Ill-formed in C++20
// In C++23 they give L['h', 'e', 'l', 'l', 'o']
std::format(L"{}", "hello"); // A libc++ bug prevents this from working.
std::format(L"{}", std::string_view("hello"));
std::format(L"{}", std::string("hello"));
std::format(L"{}", std::vector{'h', 'e', 'l', 'l', 'o'});
```

An example is shown [here](#). This only shows libc++ since libstdc++ and MSVC STL have not implemented the formatting ranges papers ([P2286R8](#) and [P2585R0](#)) yet.

The difference between C++20 and C++23 is the existence of range formatters. These formatters use the formatter specialization `formatter<char, wchar_t>` which converts the sequence of chars to a sequence of `wchar_t`s.

In this conversion `same_as<char, charT>` is false, thus the requirements of the range-type `s` and `?s` ([\[tab:formatter.range.type\]](#)) aren't met. So the following is ill-formed:

```
std::format(L"{:s}", std::string("hello")); // Not L"hello"
```

It is surprising that some string types can be formatted as a sequence of wide-characters, but others not. A sequence of characters can be a sequence UTF-8 code units. This is explicitly supported in the width estimation of string types. The conversion of `char` to `wchar_t` will convert the individual code units, which will give incorrect results for multi-byte code points. It will not transcode UTF-8 to UTF-16/32. The current behavior is not in line with the note in 22.14.6.4 [\[format.formatter.spec\]/2](#)

[Note 1: Specializations such as `formatter<wchar_t, char>` and `formatter<const char*, wchar_t>` that would require implicit multibyte / wide string or character conversion are disabled. — end note]

Disabling this could be done by explicitly disabling the `char` to `wchar_t` sequence formatter. Something along the lines of

```
template<ranges::input_range R>
requires (format_kind<R> == range_format::sequence &&
         same_as<remove_cvref_t<ranges::range_reference_t<R>>, char>)
struct formatter<R, wchar_t> : __disabled_formatter {};
```

where `__disabled_formatter` satisfies 22.14.6.4 [\[format.formatter.spec\]/5](#), would do the trick. This disables the conversion for all sequences not only the string types. So `vector`, `array`, `span`, etc. would be disabled.

This does not disable the conversion in the `range_formatter`. This allows users to explicitly opt in to this formatter for their own specializations.

An alternative would be to only disable this conversion for string type specializations (22.14.6.4 [\[format.formatter.spec\]/2.2](#)) where `char` to `wchar_t` is used:

```
template<size_t N> struct formatter<charT[N], charT>;
template<class traits, class Allocator>
struct formatter<basic_string<charT, traits, Allocator>, charT>;
```

```
template<class traits>
struct formatter<basic_string_view<charT, traits>, charT>;
```

Disabling following the following two is not strictly required:

```
template<> struct formatter<char*, wchar_t>;
template<> struct formatter<const char*, wchar_t>;
```

However, if (const) char* becomes an input_range in a future version C++, these formatters would become enabled. Disabling all five instead of the three required specializations seems like a future proof solution.

Since there is no enabled narrowing formatter specialization

```
template<> struct formatter<wchar_t, char>;
```

there are no issues for wchar_t to char conversions.

Before proceeding with a proposed resolution the following design questions need to be addressed:

- Do we want to allow string types of chars to be formatted as sequences of wchar_ts?
- Do we want to allow non string type sequences of chars to be formatted as sequences of wchar_ts?
- Should we disable char to wchar_t conversion in the range_formatter?

SG16 has indicated they would like to discuss this issue during a telecon.

[2023-06-08; Reflector poll]

Set status to SG16 and priority to 3 after reflector poll.

[2023-07-26; Mark de Wever provides wording confirmed by SG16]

[2024-03-18; Tokyo: move to Ready]

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 22.14.6.4 [\[format.formatter.spec\]](#) as indicated:

[Drafting note: The unwanted conversion happens due to the formatter base class specialization (22.14.7.3 [\[format.range.fmtdef\]](#))

```
struct range_default_formatter<range_format::sequence, R, charT>
```

which is defined the header <format>. Therefore the disabling is only needed in this header) — end drafting note]

-2- [...]

The parse member functions of these formatters interpret the format specification as a *std-format-spec* as described in 22.14.2.2 [\[format.string.std\]](#).

[Note 1: Specializations such as `formatter<wchar_t, char>` and `formatter<const char*, wchar_t>` that would require implicit multibyte / wide string or character conversion are disabled. — end note]

-?- The header <format> provides the following disabled specializations:

(?1) — The string type specializations

```
template<> struct formatter<char*, wchar_t>;
template<> struct formatter<const char*, wchar_t>;
template<size_t N> struct formatter<char[N], wchar_t>;
template<class traits, class Allocator>
struct formatter<basic_string<char, traits, Allocator>, wchar_t>;
template<class traits>
struct formatter<basic_string_view<char, traits>, wchar_t>;
```

-3- For any types T and charT for which neither the library nor the user provides an explicit or partial specialization of the class template `formatter`, `formatter<T, charT>` is disabled.

Tentatively Ready Issues

[4060](#). `submdspan` preconditions do not forbid creating invalid pointer

Section: 24.7.3.7.7 [\[mdspan.sub.sub\]](#) Status: [Tentatively Ready](#) Submitter: Mark Hoemmen Opened: 2024-03-26 Last modified: 2024-05-08

Priority: Not Prioritized

Discussion:

Oliver Lee and Ryan Wooster pointed out to us that creating a submdspan with zero-length *tuple-like* or *strided_slice* slice specifiers at the upper extent can cause the Standard submdspan_mapping overloads to access the input mdspan's mapping out of bounds. This happens in the following line of specification (24.7.3.7.6 [mdspan.sub.map] p8 in N4971 moved to [mdspan.sub.map.common] p8 after the merge of P2642R6).

Let offset be a value of type size_t equal to (*this)(first_<index_type, P>(slices...)).

If data_handle_type is a pointer to an array, then the resulting offset can be larger than required_span_size(), thus making the pointer invalid (not just one past the end). In a constexpr context, the result is ill-formed. With the [reference mdspan implementation](#), Clang can actually report a build error (e.g., for out-of-bounds access to a std::array). The contributed example illustrates this.

Oliver and Ryan offer the following example and analysis:

Example 1:

```
auto x = std::array<int, 3>{};
auto A = mdspan{x.data(), extents{3}};
auto B = submdspan(A, pair{3, 3});
```

B is an mdspan with zero elements.

Example 2:

```
auto y = std::array<int, 9>{};
auto C = mdspan{y.data(), extents{3, 3}};
auto D = submdspan(C, pair{3, 3}, pair{3, 3});
```

A precondition for each slice specifier is (24.7.3.7.5 [mdspan.sub.extents]):

$$0 \leq \text{first_<index_type, } k\text{>(slices...)} \leq \text{last_<k>(src.extents(), slices...)} \leq \text{src.extent}(k).$$

Our understanding is that precondition is satisfied. In the second example, first_<0> is 3 and first_<1> is also 3.

However, the submapping offset is defined as (*this)(first_<index_type, P>(slices...)), which then can result in an invalid data handle of the submdspan, even if the data handle is never accessed/dereferenced.

[godbolt demo](#)

We expect this situation to come up in practice.

Suppose we have an N x N mdspan representing a matrix A, and we want to partition it into a 2 x 2 "matrix of matrices" (also called a "block matrix"). This partitioning is a common operation in linear algebra algorithms such as matrix factorizations. Examples of this 2 x 2 partitioning appear in P2642 and P1673.

```
mdspan A{A_ptr, N, N};

size_t p = partition_point(N); // integer in 0, 1, ..., N (inclusive)
auto A_00 = submdspan(A, tuple{0, p}, tuple{0, p});
auto A_10 = submdspan(A, tuple{p, N}, tuple{0, 0});
auto A_01 = submdspan(A, tuple{0, p}, tuple{p, N});
auto A_11 = submdspan(A, tuple{p, N}, tuple{p, N});
```

Table illustrating the resulting 2 x 2 block matrix follows:

A_00	A_01
A_10	A_11

It's valid for p to be 0. That makes every block but A_11 have zero size. Thus, it should also be valid for p to be N. That makes every block but A_00 have zero size. However, that leads to the aforementioned UB.

It doesn't make sense to change first_ or last_. The definitions of first_ and last_ are meant to turn the slice specifier into a pair of bounds. Since submdspan(A, tuple{p, N}, tuple{p, N}) is valid even if p equals N, then that strongly suggests that first_<0> and first_<1> should always be p, even if p equals N.

As a result, we find ourselves needing to change submdspan_mapping. This will affect both the Standard submdspan_mapping overloads, and any custom (user-defined) overloads.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4971 after the merge of P2642R6.

1. Modify the new 24.7.3.7.6.1 [mdspan.sub.map.common] as indicated:

-8- If first_<index_type, k>(slices...) equals extents().extent(k) for any rank index k of extents(), then let offset be a value of type size_t equal to (*this).required_span_size(). Otherwise, let offset be a value of type size_t equal to (*this)(first_<index_type, P>(slices...)).

2. Modify 24.7.3.7.7 [mdspan.sub.sub] as indicated:

As a drive-by readability fix, we also propose changing a variable name in paragraph 6 as indicated below.

```
template<class ElementType, class Extents, class LayoutPolicy,
        class AccessorPolicy, class... SliceSpecifiers>
constexpr auto submdspan(
    const mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>& src,
    SliceSpecifiers... slices) -> see below;
```

-1- Let `index_type` be `typename Extents::index_type`.

-2- Let `sub_map_offset` be the result of `submdspan_mapping(src.mapping(), slices...)`.

[...]

-3- *Constraints*: [...]

-4- *Mandates*: [...]

-5- *Preconditions*: [...]

-6- *Effects*: Equivalent to:

```
auto sub_map_result_offset = submdspan_mapping(src.mapping(), slices...);
return mdspan(src.accessor().offset(src.data(), sub_map_result_offset.offset),
    sub_map_result_offset.mapping,
    AccessorPolicy::offset_policy(src.accessor()));
```

[4061](#). Should `std::basic_format_context` be default-constructible/copyable/movable?

Section: 22.14.6.7 [\[format.context\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2024-03-24 **Last modified:** 2024-03-26

Priority: Not Prioritized

View all other [issues](#) in `[format.context]`.

Discussion:

Per 22.14.6.7 [\[format.context\]](#), it seems that `std::basic_format_context` has a default constructor that is effectively defaulted, which means that it is default constructible if and only if `OutIt` is default constructible. Currently only `libstdc++` makes it conditionally default constructible, while `libc++` and `MSVC STL` (together with `fmtlib`) make it never default constructible.

It seems that `basic_format_context` objects are supposed to be created by the implementation in some internal way, and user codes are only supposed to modify existing `basic_format_context` objects during formatting.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4971](#).

1. Modify 22.14.6.7 [\[format.context\]](#) as indicated:

```
namespace std {
    template<class Out, class charT>
    class basic_format_context {
        basic_format_arg<basic_format_context> args_; // exposition only
        Out out_; // exposition only

        basic_format_context(const basic_format_context&) = delete;
        basic_format_context& operator=(const basic_format_context&) = delete;
    public:
        using iterator = Out;
        using char_type = charT;
        template<class T> using formatter_type = formatter<T, charT>;

        basic_format_arg<basic_format_context> arg(size_t id) const noexcept;
        std::locale locale();

        iterator out();
        void advance_to(iterator it);
    };
}
```

[4071](#). `reference_wrapper` comparisons are not SFINAE-friendly

Section: 22.10.6.6 [\[refwrap.comparisons\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2024-04-19 **Last modified:** 2024-04-24

Priority: Not Prioritized

Discussion:

[P2944R3](#) added these hidden friends to `reference_wrapper`:

```
friend constexpr synth-three-way-result<T> operator<==>(reference_wrapper, reference_wrapper);
friend constexpr synth-three-way-result<T> operator<==>(reference_wrapper, const T&);
friend constexpr synth-three-way-result<T> operator<==>(reference_wrapper, reference_wrapper<const T>);
```

These functions are not templates, and so their declarations are ill-formed for any type that does have any comparison operators, e.g.

```
struct A { } a;
std::reference_wrapper<A> r(a);
```

Instantiating `reference_wrapper<A>` will instantiate the declarations of the hidden friends, which will attempt to determine the return types of the `operator<==>` functions. That fails because `synth-three-way` is constrained and can't be called with arguments of type `A`.

This can be solved by changing those functions into templates, so they aren't instantiated eagerly, e.g.,

```
template<class U = T>
friend constexpr synth-three-way-result<U> operator<==>(reference_wrapper, reference_wrapper);
```

or by giving them a deduced return type (so that it isn't instantiated eagerly) and constraining them to only be callable when valid:

```
friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper x, reference_wrapper y)
requires requires (const T t) { synth-three-way(t, t); }
```

The second alternative is used in the proposed resolution.

In practice the *requires*-clause can be implemented more simply (and efficiently) by checking the constraints of `synth-three-way` directly:

```
requires (const T t) { { t < t } -> boolean-testable; }
```

but when specified in prose in a *Constraints*: element it seems clearer to just use `synth-three-way(x.get(), y.get())`.

The proposed resolution has been committed to libstdc++'s master branch.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 22.10.6.1 [\[refwrap.general\]](#) as indicated:

```
// [refwrap.comparisons], comparisons
friend constexpr bool operator==(reference_wrapper, reference_wrapper);
friend constexpr bool operator==(reference_wrapper, const T&);
friend constexpr bool operator==(reference_wrapper, reference_wrapper<const T>);

friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper, reference_wrapper);
friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper, const T&);
friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper, reference_wrapper<const T>);
```

2. Modify 22.10.6.6 [\[refwrap.comparisons\]](#) as indicated:

```
friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper x, reference_wrapper y);

-?- Constraints: The expression synth-three-way(x.get(), y.get()) is well-formed.

-?- Returns: synth-three-way(x.get(), y.get()).

friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper x, const T& y);

-?- Constraints: The expression synth-three-way(x.get(), y) is well-formed.

-?- Returns: synth-three-way(x.get(), y).

friend constexpr synth-three-way-result<T> auto operator<==>(reference_wrapper x, reference_wrapper<const T> y);

-?- Constraints: is_const_v<T> is false. The expression synth-three-way(x.get(), y.get()) is well-formed.

-?- Returns: synth-three-way(x.get(), y.get()).
```

Priority: Not Prioritized

View other [active issues](#) in [\[range.join.with.view\]](#).

View all other [issues](#) in [\[range.join.with.view\]](#).

Discussion:

`join_with_view` requires the value type, reference and rvalue reference of the inner range and pattern range to share common (reference) types through `compatible-joinable-ranges`.

However, unlike what `concat_view` and `generator` do, this concept only requires that these three types be valid and does not further check the relationship between them to be compatible with the `indirectly_readable` requirement for `input_iterator`. This results in a validly-constructed `join_with_view` that may not model `input_range`, which seems unintended.

The proposed resolution aliases `compatible-joinable-ranges` to `concatable` i.e. specialization for two ranges to fully constrain, and I believe this could also be a better fit for LWG [3971](#).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4981](#).

1. Modify 26.7.15.2 [\[range.join.with.view\]](#) as indicated:

```
namespace std::ranges {
    template<class R, class P>
        concept compatible-joinable-ranges = concatable<R, P>; // exposition only
        common_with<range_value_t<R>, range_value_t<P>>> &&
        common_reference_with<range_reference_t<R>, range_reference_t<P>>> &&
        common_reference_with<range_rvalue_reference_t<R>, range_rvalue_reference_t<P>>> &&
    [...]
}
```

[2024-04-24; Hewill Kang provides improved wording]

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 26.2 [\[ranges.syn\]](#) as indicated:

```
#include <compare> // see 17.11.1 \[compare.syn\]
#include <initializer_list> // see 17.10.2 \[initializer.list.syn\]
#include <iterator> // see 25.2 \[iterator.synopsis\]

namespace std::ranges {
    [...]
    // 26.7.15 \[range.join.with\], join with view
    template<class R, class P>
    concept compatible joinable ranges = see below; // exposition only

    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
        && view<Pattern>
        && compatible joinable ranges<range_reference_t<V>, Pattern>
        see below
    class join_with_view; // freestanding
    [...]
}
```

2. Modify 26.7.15.2 [\[range.join.with.view\]](#) as indicated:

```
namespace std::ranges {
    template<class R, class P>
    concept compatible joinable ranges = // exposition only
    common_with<range_value_t<R>, range_value_t<P>>> &&
    common_reference_with<range_reference_t<R>, range_reference_t<P>>> &&
    common_reference_with<range_rvalue_reference_t<R>, range_rvalue_reference_t<P>>> &&
    [...]

    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
        && view<Pattern>
        && compatible joinable rangesconcatable<range_reference_t<V>, Pattern>
    class join_with_view : public view_interface<join_with_view<V, Pattern>> {
        [...]
        constexpr auto begin() const
    }
```

```

        requires forward_range<const V> &&
               forward_range<const Pattern> &&
               is_reference_v<range_reference_t<const V>> &&
               input_range<range_reference_t<const V>> &&
               concatable<range_reference_t<const V>, const Pattern> {
            return iterator<true>{*this, ranges::begin(base_)};
        }
        [...]
    constexpr auto end() const
    requires forward_range<const V> && forward_range<const Pattern> &&
             is_reference_v<range_reference_t<const V>> &&
             input_range<range_reference_t<const V>> &&
             concatable<range_reference_t<const V>, const Pattern> {
        [...]
    }
};
}

```

3. Modify 26.7.15.3 [\[range.join.with.iterator\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
    requires view<V> && input_range<range_reference_t<V>>
             && view<Pattern> && compatible_joinable_rangesconcatable<range_reference_t<V>, Pattern>
    template<bool Const>
    class join_with_view<V, Pattern>::iterator {
        [...]
    };
}

```

4. Modify 26.7.15.4 [\[range.join.with.sentinel\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
    requires view<V> && input_range<range_reference_t<V>>
             && view<Pattern> && compatible_joinable_rangesconcatable<range_reference_t<V>, Pattern>
    template<bool Const>
    class join_with_view<V, Pattern>::sentinel {
        [...]
    };
}

```

4076. concat_view should be freestanding

Section: 17.3.2 [\[version.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2024-04-21 **Last modified:** 2024-04-21

Priority: Not Prioritized

View other [active issues](#) in [\[version.syn\]](#).

View all other [issues](#) in [\[version.syn\]](#).

Discussion:

concat_view can be freestanding, but this never seems to come up in the discussion, which seems to be an oversight.

[2024-04-21; Daniel comments]

The specification of some member functions of concat_view seem to depend on freestanding-deleted get overloads for variant, but so does join_with_view, which is marked as freestanding, so it does not seem to be a good reason to accept join_with_view but not concat_view as freestanding.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 17.3.2 [\[version.syn\]](#) as indicated:

```

#define __cpp_lib_ranges_concat 202403L // freestanding, also in <ranges>

```

2. Modify 26.2 [\[ranges.syn\]](#) as indicated:

```

#include <compare> // see 17.11.1 \[compare.syn\]
#include <initializer_list> // see 17.10.2 \[initializer.list.syn\]
#include <iterator> // see 25.2 \[iterator.synopsis\]

namespace std::ranges {
    [...]
    // 26.7.18 \[range.concat\], concat view
    template<input_range... Views>
    requires see below
    class concat_view;
}

```

// freestanding

```
namespace views { inline constexpr unspecified concat = unspecified; } // freestanding
[...]
```

4079. Missing *Preconditions* in `concat_view::iterator`'s conversion constructor

Section: 26.7.18.3 [\[range.concat.iterator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2024-04-26 **Last modified:** 2024-04-27

Priority: Not Prioritized

View other [active issues](#) in `[range.concat.iterator]`.

View all other [issues](#) in `[range.concat.iterator]`.

Discussion:

This conversion constructor obtains the alternative iterator of the argument through `std::get`, which will throw when the variant is valueless. We seem to be missing a *Preconditions* element here.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 26.7.18.3 [\[range.concat.iterator\]](#) as indicated:

```
constexpr iterator(iterator<!Const> it)
requires Const &&
    (convertible_to<iterator_t<Views>, iterator_t<const Views>> && ...);
```

-?- *Preconditions:* `it.it.valueless()` is false.

-8- *Effects:* Initializes `parent_` with `it.parent_`, and let `i` be `it.it.index()`, initializes `it_` with `base-iter(in_place_index<i>, std::get<i>(std::move(it.it)))`.

4082. `views::concat(r)` is well-formed when `r` is an `output_range`

Section: 26.7.18.1 [\[range.concat.overview\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2024-04-27 **Last modified:** 2024-04-28

Priority: Not Prioritized

Discussion:

Currently, `views::concat` will return `views::all(r)` when it takes only one argument, which only requires that the type of `r` models `viewable_range` which includes `output_range`:

```
std::vector<int> v;
auto r = std::views::counted(std::back_inserter(v), 3);
auto c = std::views::concat(r); // well-formed
```

Since `concat_view` requires all ranges to be `input_range`, this seems inconsistent. We should reject the above just like `views::zip_transform` still requires `F` to be `move_constructible` in the case of an empty pack.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 26.7.18.1 [\[range.concat.overview\]](#) as indicated:

-2- The name `views::concat` denotes a customization point object (16.3.3.3.5 [\[customization.point.object\]](#)). Given a pack of subexpressions `Es...`, the expression `views::concat(Es...)` is expression-equivalent to

(2.1) — `views::all(Es...)` if `Es` is a pack with only one element [whose type models input_range](#).

(2.2) — otherwise, `concat_view(Es...)`.

4083. `views::as_rvalue` should reject non-input ranges

Section: 26.7.7.1 [\[range.as.rvalue.overview\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2024-04-27 **Last modified:** 2024-04-28

Priority: Not Prioritized

View other [active issues](#) in [range.as.rvalue.overview].

View all other [issues](#) in [range.as.rvalue.overview].

Discussion:

views::as_rvalue(r) equivalent to views::all(r) when r's reference and rvalue reference are of the same type, which means that in this case we only need to check whether the type of r models viewable_range.

However, libstdc++'s implementation always requires as_rvalue_view{r} to be valid, which leads to divergence when r is not an input_range ([demo](#)):

```
#include <ranges>

struct I {
    int operator*();
    using difference_type = int;
    I& operator++();
    void operator++(int);
};

std::ranges::range auto r = std::ranges::subrange{I{}, std::unreachable_sentinel}
    | std::views::as_rvalue; // // well-formed in libc++/MSVC-STL, ill-formed in libstdc++
```

Although this is precisely a bug in libstdc++ that does not conform to the current wording, it is reasonable to require r to be an input_range to be consistent with the constraints of as_rvalue_view.

[2024-05-08; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 26.7.7.1 [\[range.as.rvalue.overview\]](#) as indicated:

-2- The name views::as_rvalue denotes a range adaptor object (26.7.2 [\[range.adaptor.object\]](#)). Let E be an expression and let T be decltype((E)). The expression views::as_rvalue(E) is expression-equivalent to:

(2.1) — views::all(E) if **T models input_range and** same_as<range_rvalue_reference_t<T>, range_reference_t<T>> is true.

(2.2) — Otherwise, as_rvalue_view(E).

[4096](#). views::iota(views::iota(0)) should be rejected

Section: 26.6.4.1 [\[range.iota.overview\]](#) Status: [Tentatively Ready](#) Submitter: Hewill Kang Opened: 2024-05-08 Last modified: 2024-05-09

Priority: Not Prioritized

Discussion:

views::iota(E) literally means incrementing element E endlessly, but views::iota(views::iota(0)) is currently well-formed due to CTAD, rejecting such unreasonable spelling seems therefore reasonable.

[2024-06-24; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 26.6.4.1 [\[range.iota.overview\]](#) as indicated:

-1- iota_view generates a sequence of elements by repeatedly incrementing an initial value.

-2- The name views::iota denotes a customization point object (16.3.3.3.5 [\[customization.point.object\]](#)). Given subexpressions E and F, the expressions views::iota(E) and views::iota(E, F) are expression-equivalent to iota_view<decay_t<decltype((E))>>(E) and iota_view(E, F), respectively.

[4098](#). views::adjacent<0> should reject non-forward ranges

Section: 26.7.27.1 [\[range.adjacent.overview\]](#), 26.7.28.1 [\[range.adjacent.transform.overview\]](#) Status: [Tentatively Ready](#) Submitter: Hewill Kang Opened: 2024-05-10 Last modified: 2024-06-24

Priority: Not Prioritized

View all other [issues](#) in [range.adjacent.overview].

Discussion:

Following-up LWG 4082 and LWG 4083, the current wording makes `views::adjacent<0>(r)` and `views::adjacent_transform<0>(r, [] { return 0; })` well-formed even when `r` is just an input range or an output range, which seems to be an oversight.

[2024-06-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 26.7.27.1 [\[range.adjoint.overview\]](#) as indicated:

-2- The name `views::adjacent<N>` denotes a range adaptor object (26.7.2 [\[range.adaptor.object\]](#)). Given a subexpression `E` and a constant expression `N`, the expression `views::adjacent<N>(E)` is expression-equivalent to

(2.1) — `((void)E, auto{views::empty<tuple<>>})` if `N` is equal to `0` **and `decltype((E))` models forward range**.

(2.2) — otherwise, `adjacent_view<views::all_t<decltype((E))>, N>(E)`.

2. Modify 26.7.28.1 [\[range.adjoint.transform.overview\]](#) as indicated:

-2- The name `views::adjacent_transform<N>` denotes a range adaptor object (26.7.2 [\[range.adaptor.object\]](#)). Given subexpressions `E` and `F` and a constant expression `N`:

(2.1) — If `N` is equal to `0` **and `decltype((E))` models forward range**, `views::adjacent_transform<N>(E, F)` is expression-equivalent to `((void)E, views::zip_transform(F))`, except that the evaluations of `E` and `F` are indeterminately sequenced.

(2.2) — Otherwise, the expression `views::adjacent_transform<N>(E, F)` is expression-equivalent to `adjacent_transform_view<views::all_t<decltype((E))>, decay_t<decltype((F))>, N>(E, F)`.

4105. `ranges::ends_with`'s *Returns* misses difference casting

Section: 27.6.17 [\[alg.ends.with\]](#) Status: [Tentatively Ready](#) Submitter: Hewill Kang Opened: 2024-05-17 Last modified: 2024-05-19

Priority: Not Prioritized

Discussion:

The *Returns* of the `ranges` version of `ranges::ends_with` are specified as `ranges::equal(ranges::drop_view(ranges::ref_view(r1), N1 - N2), r2, ...)` which is not quite right when `N2` is an integer-class type and `N1` is an integer type, because in this case `N1 - N2` will be an integer-class type which cannot be implicitly converted to the difference_type of `r1` leading to the construction of `drop_view` being ill-formed.

[2024-06-24; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 27.6.17 [\[alg.ends.with\]](#) as indicated:

```
template<input_range R1, input_range R2, class Pred = ranges::equal_to, class Proj1 = identity,
        class Proj2 = identity>
requires (forward_range<R1> || sized_range<R1>) &&
         (forward_range<R2> || sized_range<R2>) &&
         indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::ends_with(R1&& r1, R2&& r2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});
```

-3- Let `N1` be `ranges::distance(r1)` and `N2` be `ranges::distance(r2)`.

-4- *Returns*: false if `N1 < N2`, otherwise

`ranges::equal(ranges::drop_view(ranges::ref_view(r1), N1 - static_cast<decltype(N1)>(N2)), r2, pred, proj1, proj2)`

4106. `basic_format_args` should not be default-constructible

Section: 22.14.8.3 [\[format.args\]](#) Status: [Tentatively Ready](#) Submitter: Hewill Kang Opened: 2024-05-17 Last modified: 2024-05-19

Priority: Not Prioritized

View all other [issues](#) in `[format.args]`.

Discussion:

It's unclear why `basic_format_args` originally provided a default constructor and its actual use cases, all three major libraries declare them as default, which allows value initialization of `basic_format_args` at compile time.

This does not fully conform to the current wording as its signature does not have the constexpr specifier.

Additionally, the current wording only initializes the size_ member in the default constructor, which may lead to undefined behavior when copying basic_format_args as uninitialized data_ member is copied. There is also an implementation divergence ([demo](#)):

```
#include <format>
constexpr std::format_args fmt_args; // only well-formed in MSVC-STL
```

One option is to add default member initializers for all members and default the default constructor to best match the status quo, which guarantees that basic_format_args is constexpr-able.

However, given that basic_format_args has different implementation details in the three libraries, its actual members may not be size_ and data_. It is unnecessary to ensure that all the internal members are initialized when default-constructed basic_format_args, indicating that not providing one is reasonable.

The proposed solution is to prefer the more aggressive one.

[2024-05-19; Daniel comments]

The here suggested proposal to remove the default constructor implicitly depends on the decision of LWG [4061](#) to remove basic_format_context's default constructor, since its usage would require that the exposition-only member args_ of type basic_format_args<basic_format_context> can be default-constructed as well.

[2024-06-24; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4981](#).

1. Modify 22.14.8.3 [\[format.args\]](#) as indicated:

```
namespace std {
    template<class Context>
    class basic_format_args {
        size_t size_;           // exposition only
        const basic_format_arg<Context>* data_; // exposition only

    public:
        basic_format_args() noexcept;

        template<class... Args>
        basic_format_args(const format_arg_store<Context, Args...>& store) noexcept;

        basic_format_arg<Context> get(size_t i) const noexcept;
    };
    [...]
}

[...]
```

~~basic_format_args() noexcept;~~

~~2- Effects Initializes size_ with 0.~~